

How tracking estimates changed the way I work

Lessons learned and strategies to improve

Thomas Aglassinger, 2026-05-30

© 2026 and licensed under [CC BY-SA 4.0](#).



Siisurit



Barcamp Graz 2026

About me

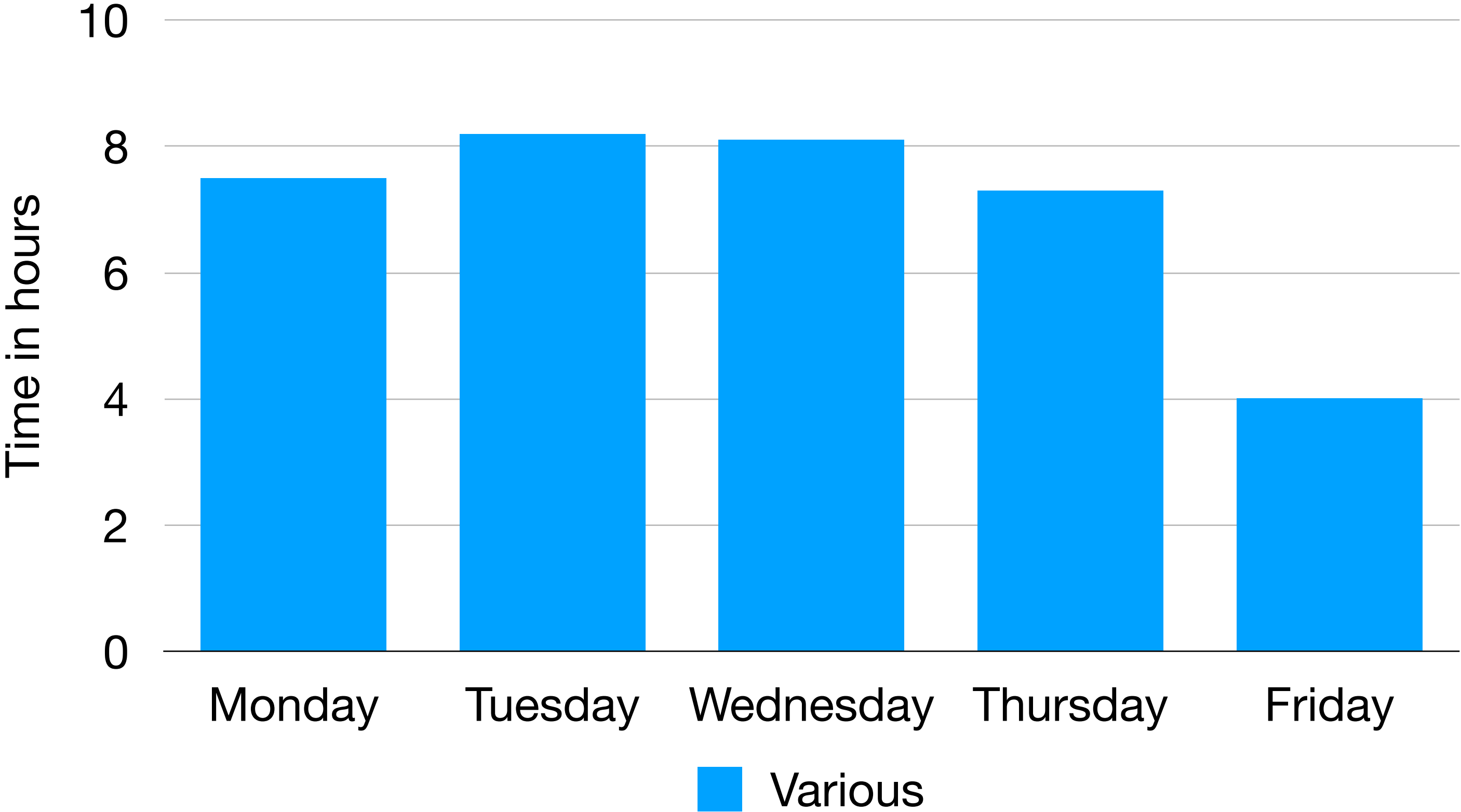
Thomas Aglassinger

- Software developer
 - 25 years of industry experience
 - in various sectors: finance, health, e-commerce, ...
- Part-time employed at Providens Analytics GmbH: <https://www.providens.at/>
- Freelancing and founder of Siisurit e.U. <https://siisurit.com/>
- Casual open source developer: <https://roskakori.at/>
- Co-organizer of Python user group Graz: <https://pygraz.org/>
- Leitmotiv: Collect, process, visualize data & analyze and optimize process

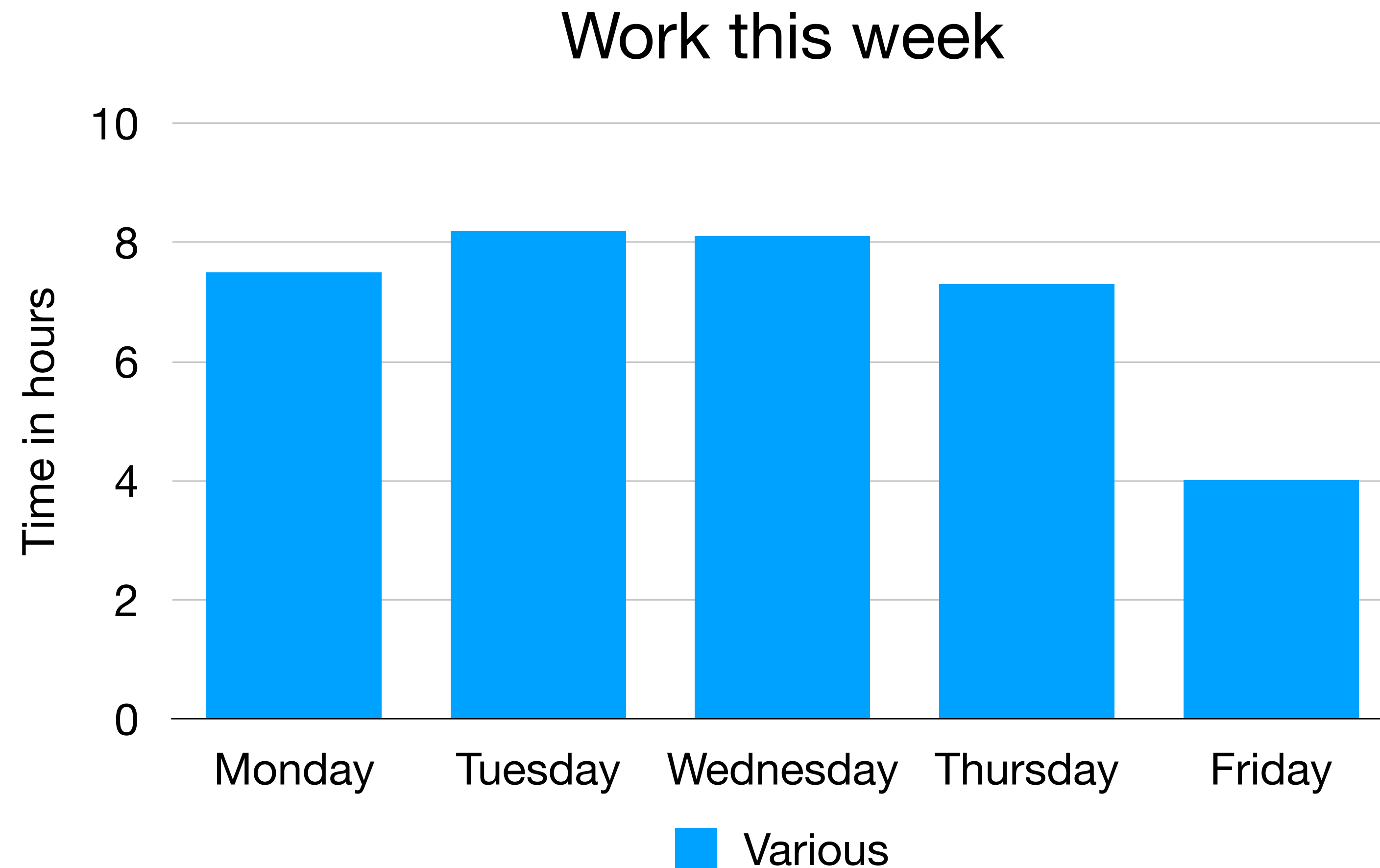
Tracking time

Track time: presence

Work this week



Track time: presence

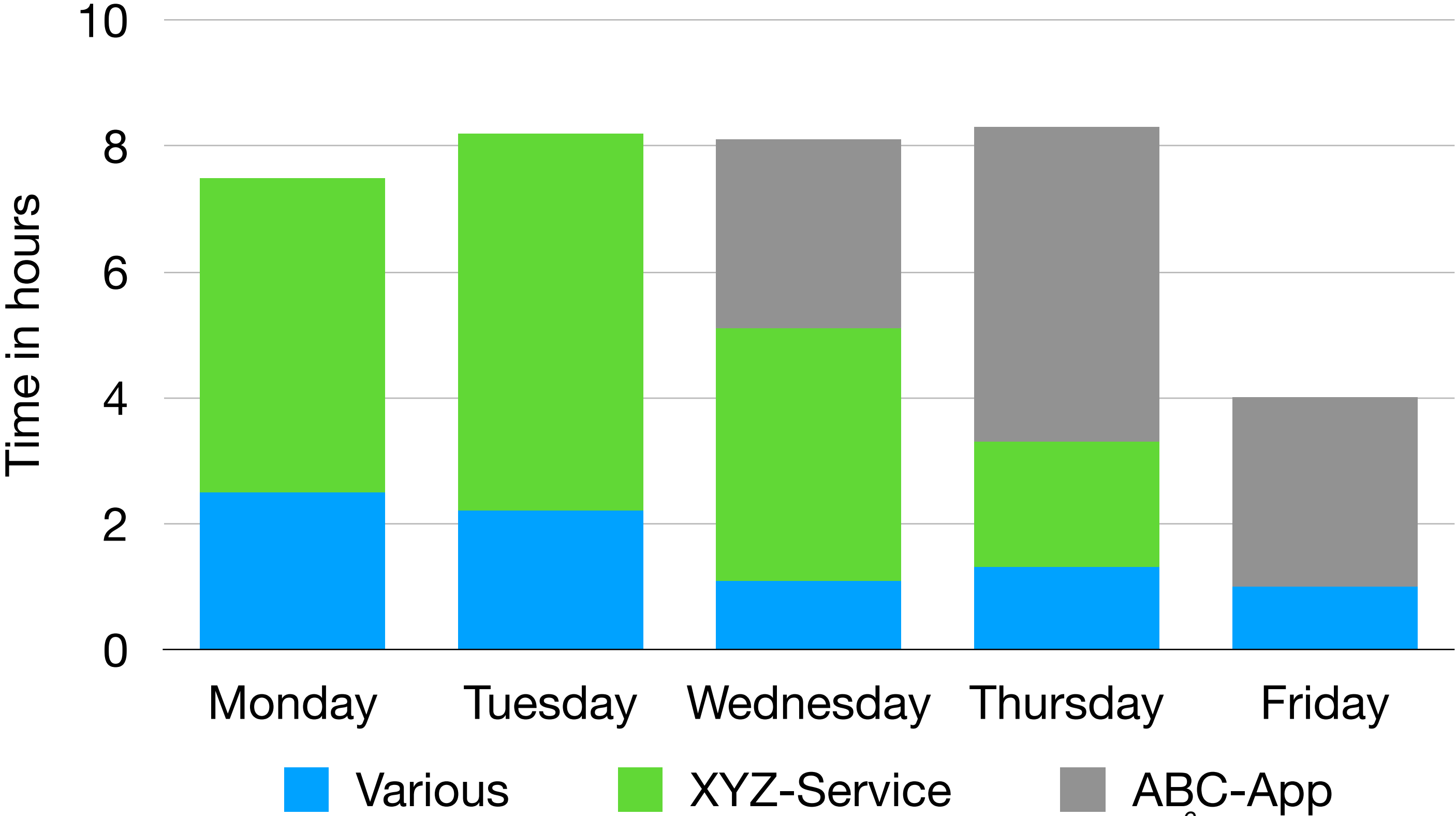


Useful to:

- Pay salary
- Who is present today?
- Do people work the agreed amount of time?

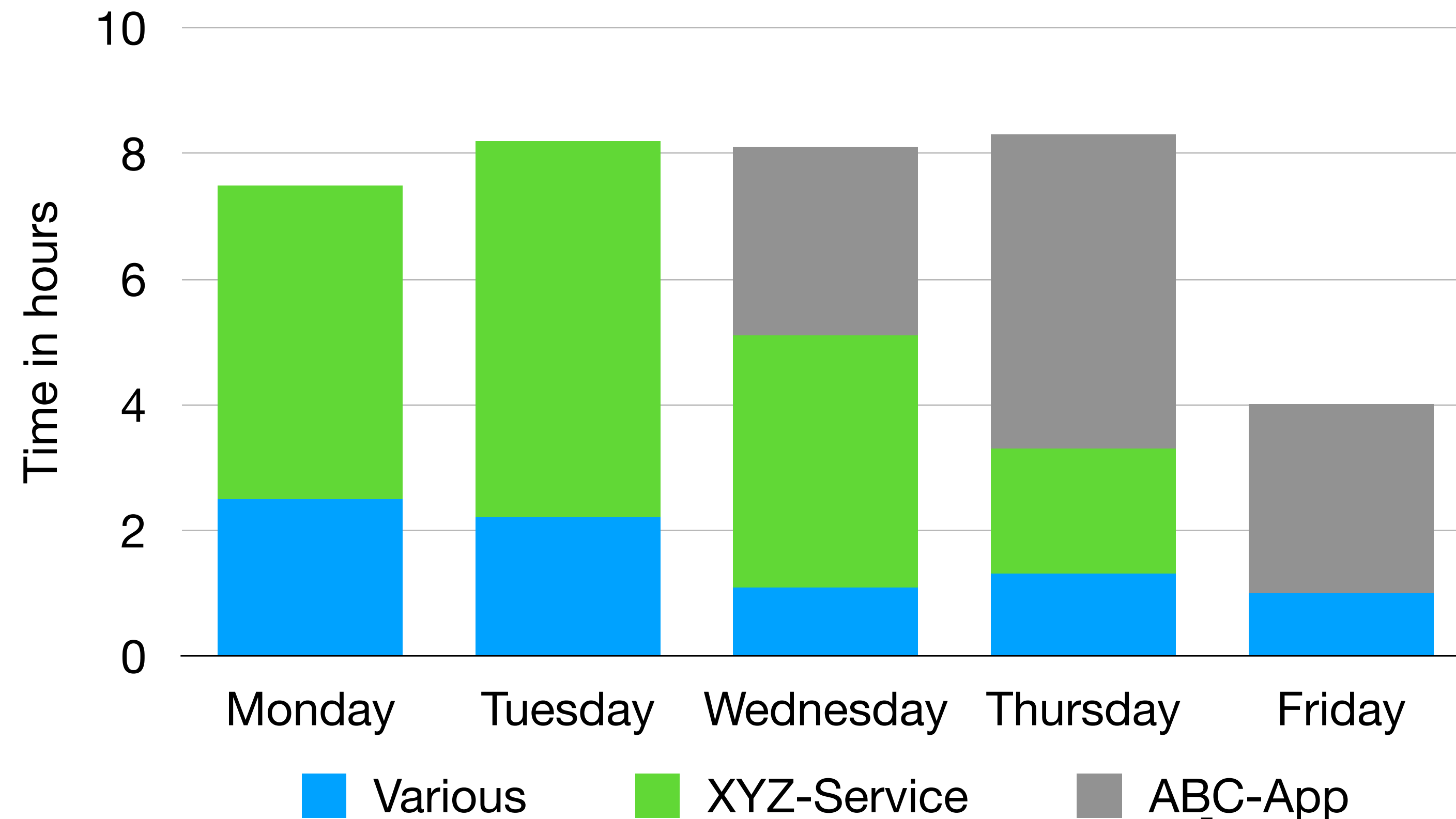
Track time: projects

Work this week



Track time: projects

Work this week

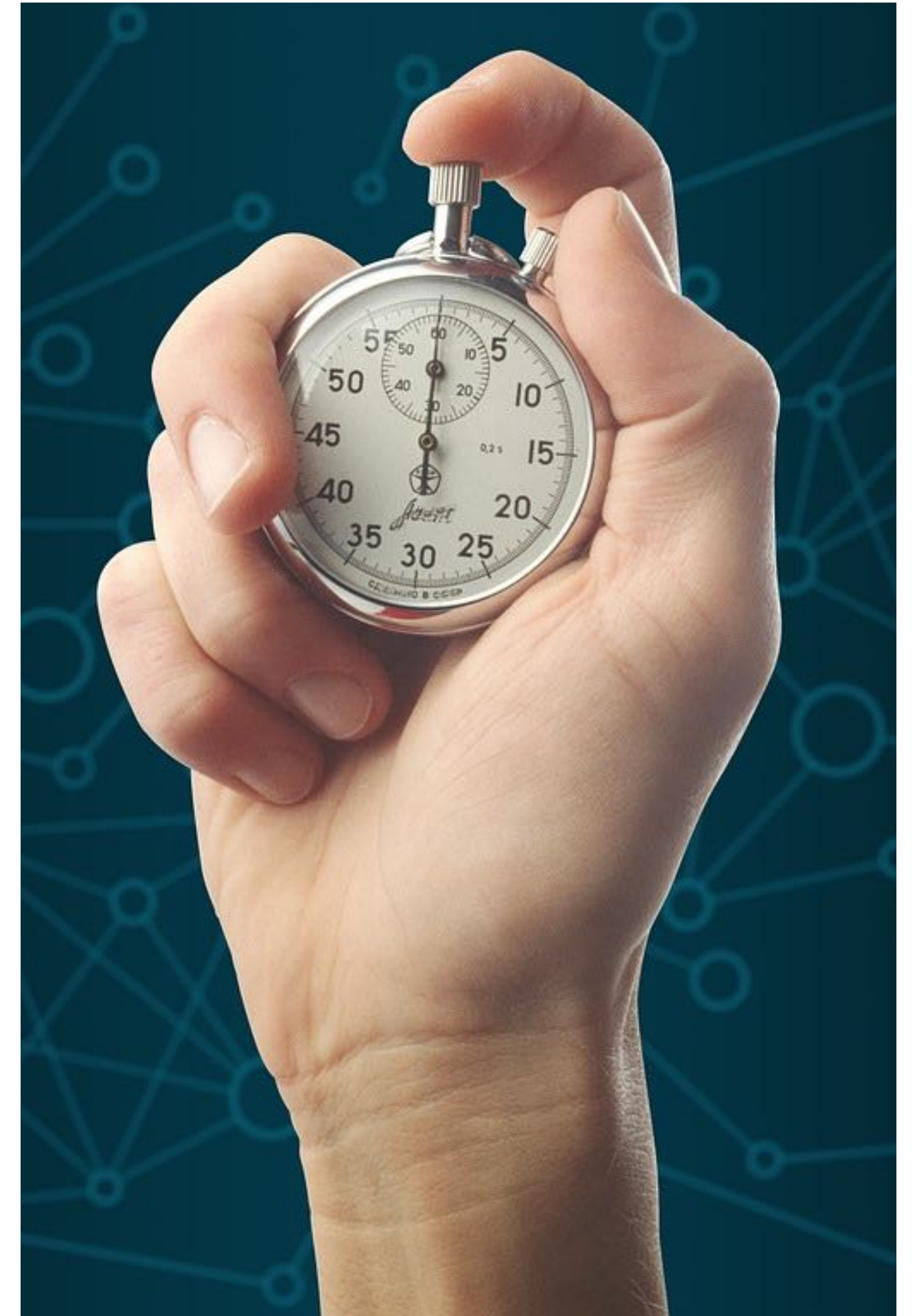


Useful to:

- Bill projects
- How much do we work on paid projects, and how much on "other things"?

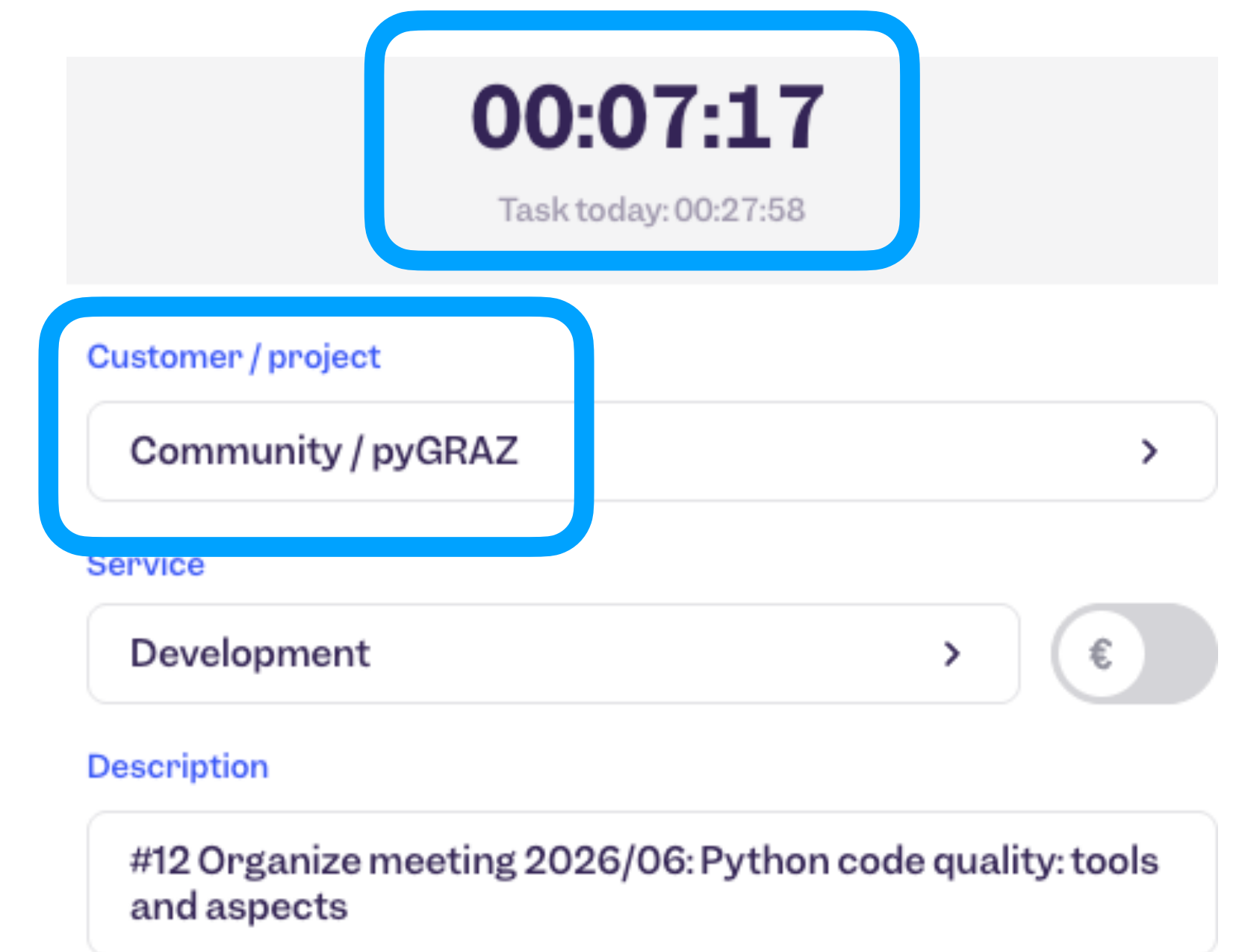
Tracking time

- Many commercial tools available depending on your exact needs:
 - Local laws?
 - Special tools including: Budgeting? Driver's logbook? ...
 - Integration with existing tools in my organization?
- Some open source, for example:
 - TimeTagger <https://timetagger.app/>
 - Kimai <https://www.kimai.org/> (with commercial extensions)



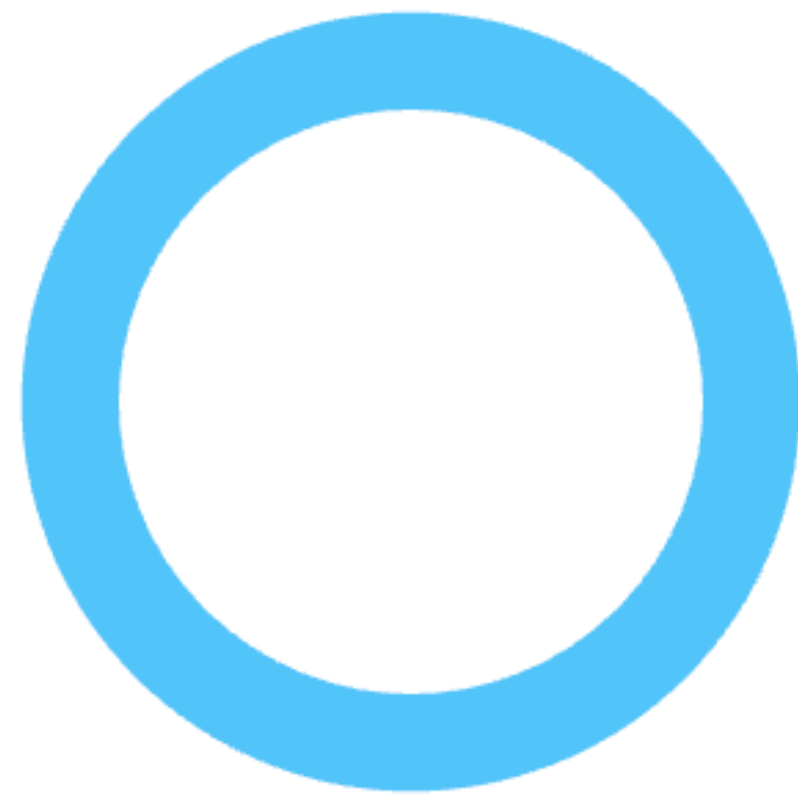
Tracking time

- Many commercial tools available depending on your exact needs:
 - Local laws?
 - Special tools including: Budgeting? Driver's logbook? ...
 - Integration with existing tools in my organization?
- Some open source, for example:
 - TimeTagger <https://timetagger.app/>
 - Kimai <https://www.kimai.org/> (with commercial extensions)

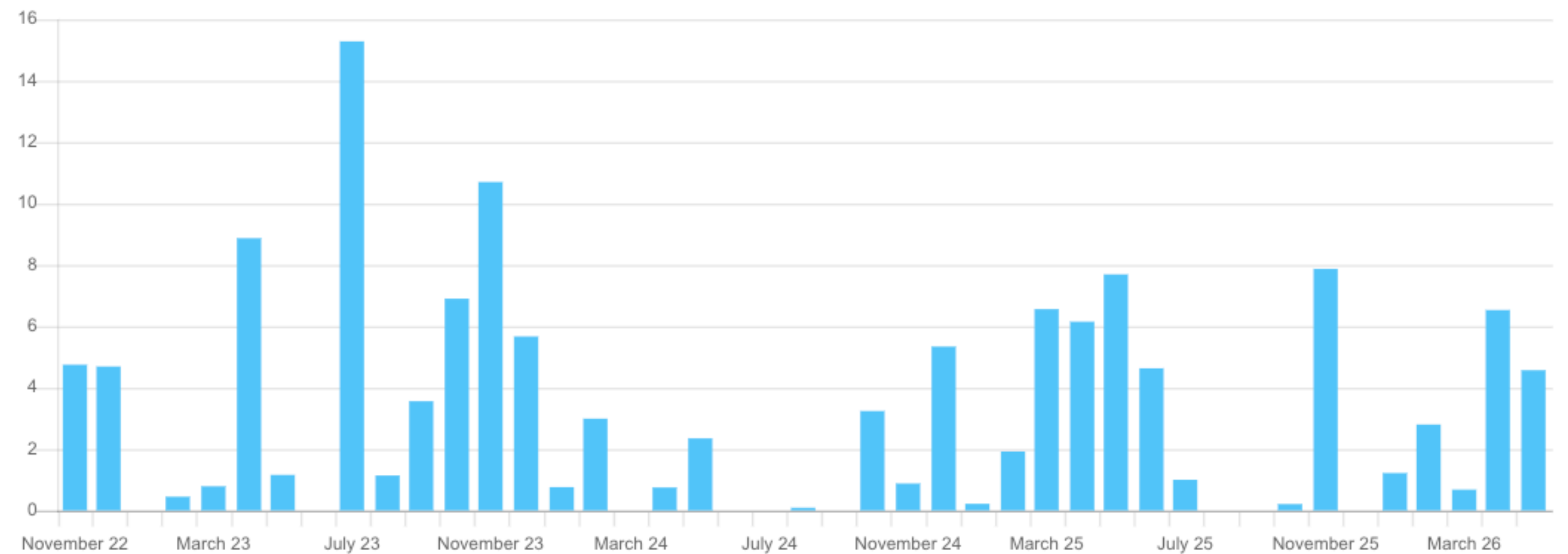


Example: Python user group Graz

Total: 132h from Nov 22 to Mai 26 = 3h / month



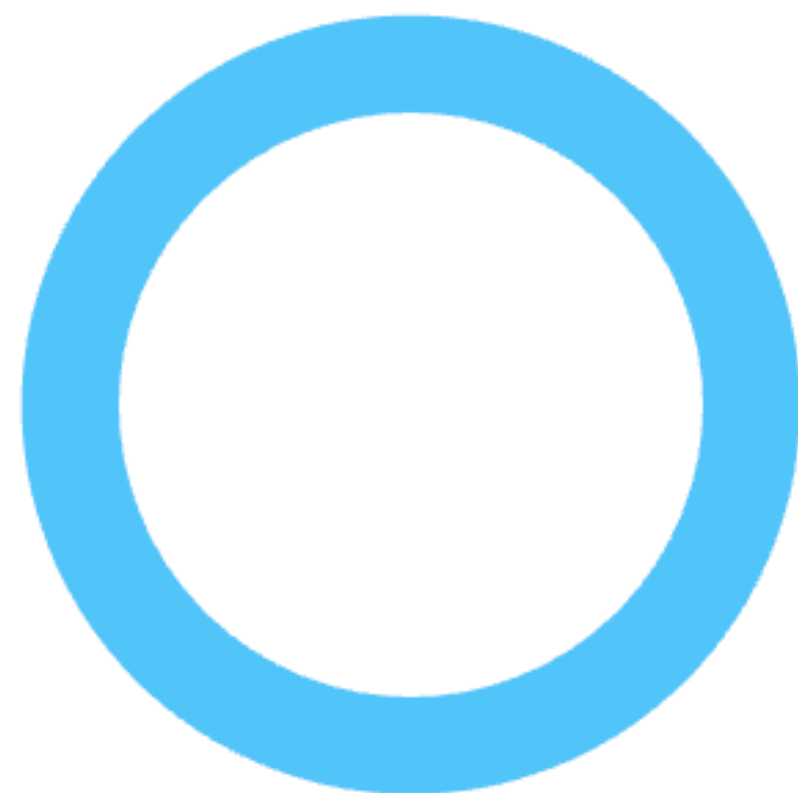
Community / pyG...



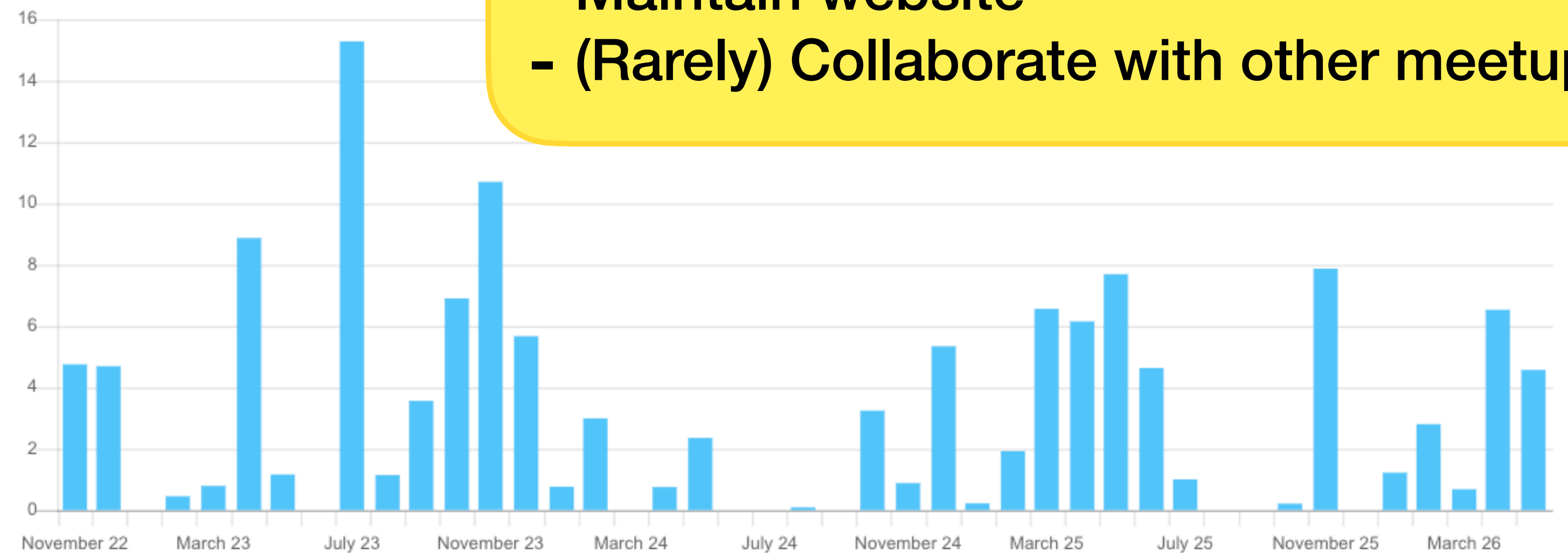
Example: Python user group Graz

What is the time spent on?

- Organize meetings
- Learn from others
- Have fun & network
- Occasionally give talk
- Maintain website
- (Rarely) Collaborate with other meetups



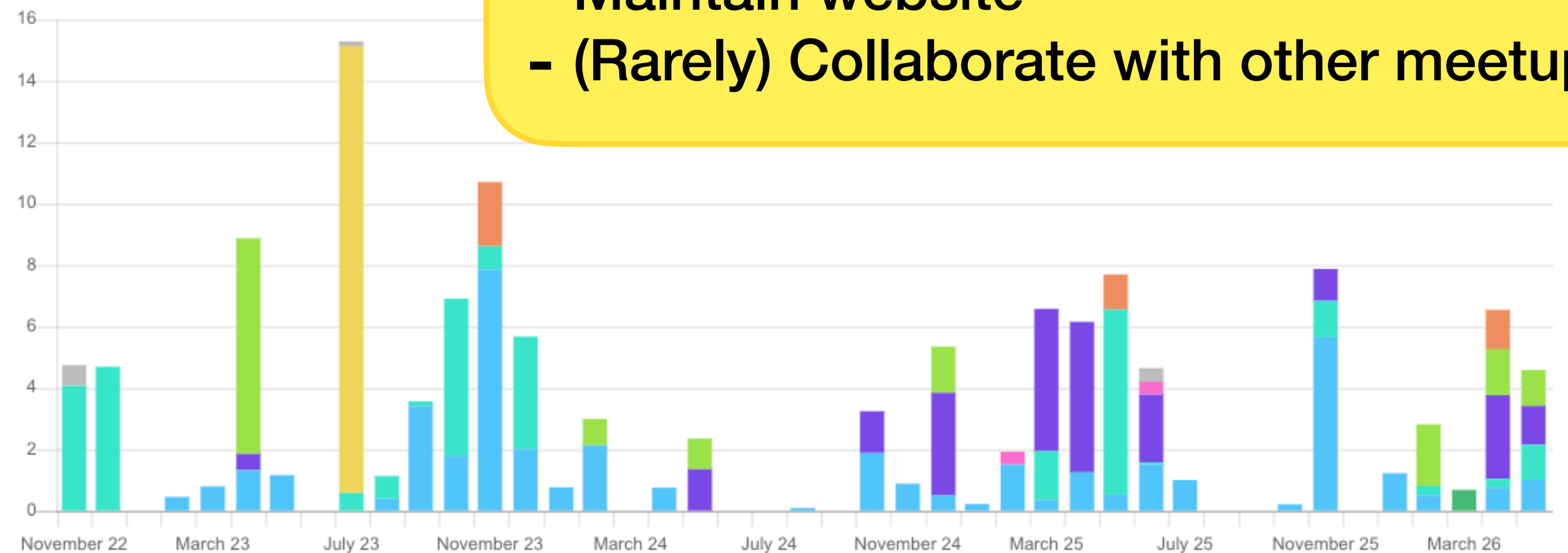
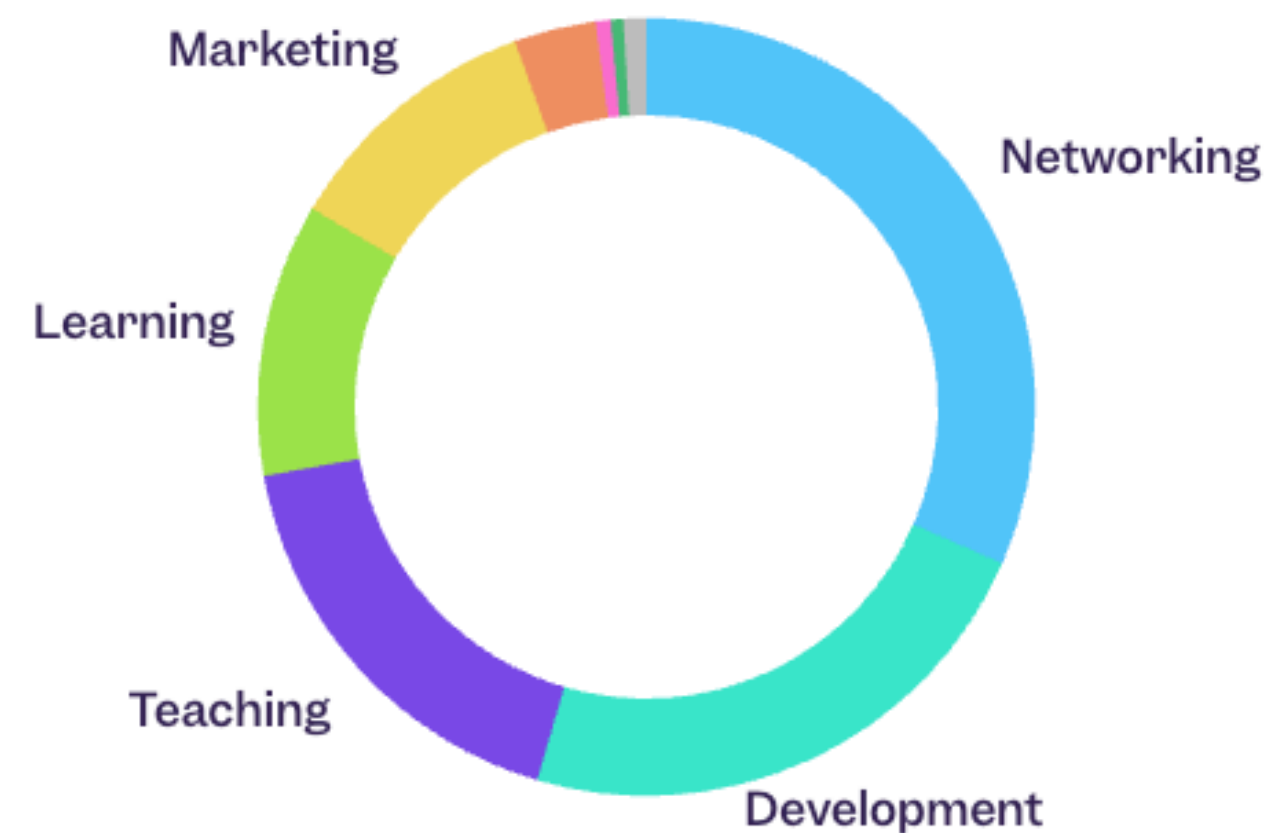
Community / pyG...



Example: Python user group Graz

Group time by activity / service / ...

- Organize meetings
- Learn from others
- Have fun & network
- Occasionally give talk
- Maintain website
- (Rarely) Collaborate with other meetups



Example: Python user group Graz

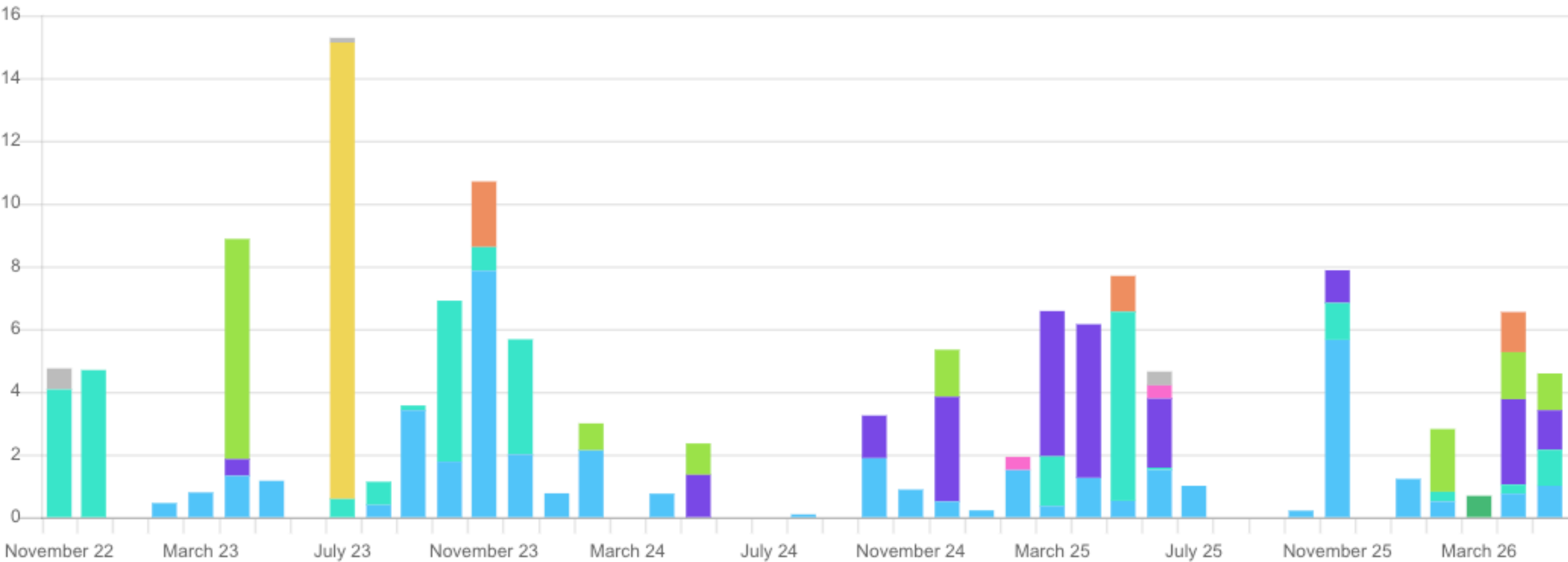
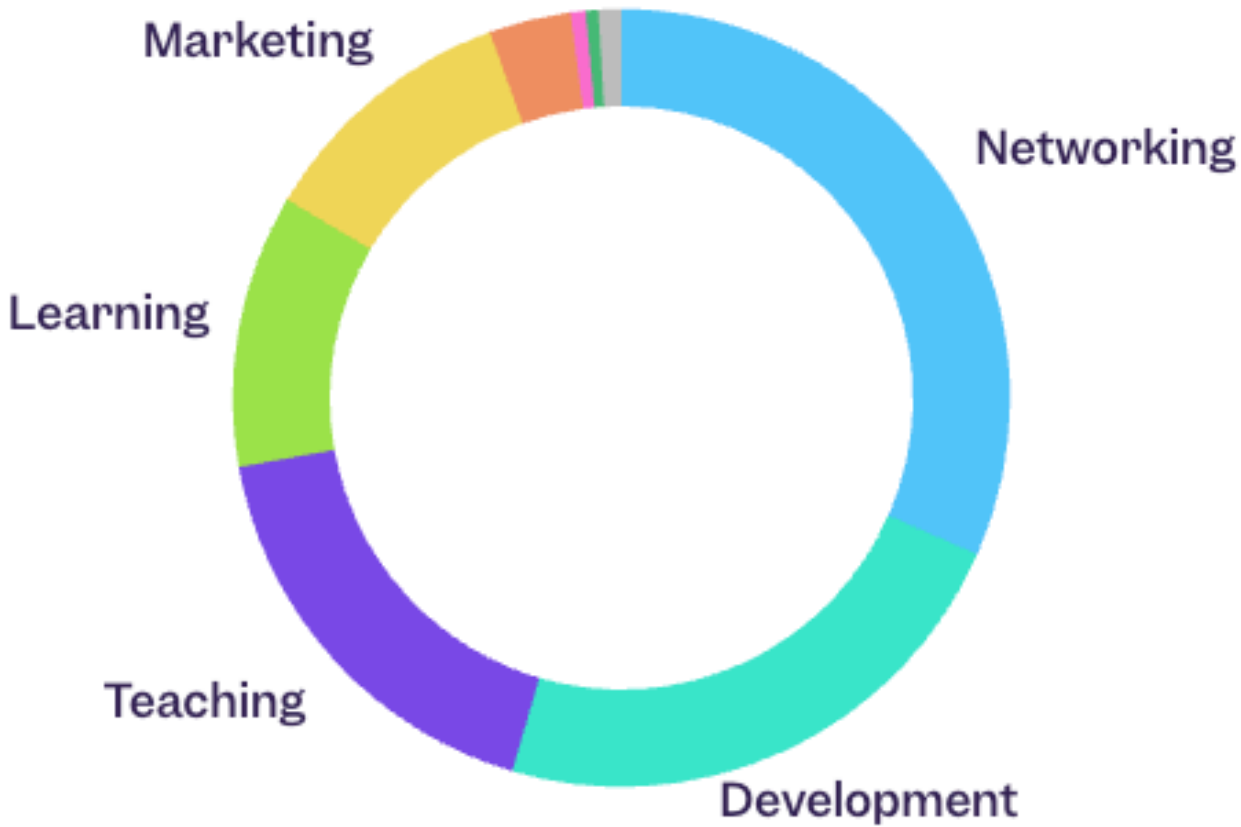
Learnings from time by activity

Collaboration with UX Graz

Prepare+give lightning talk = 1- 4h

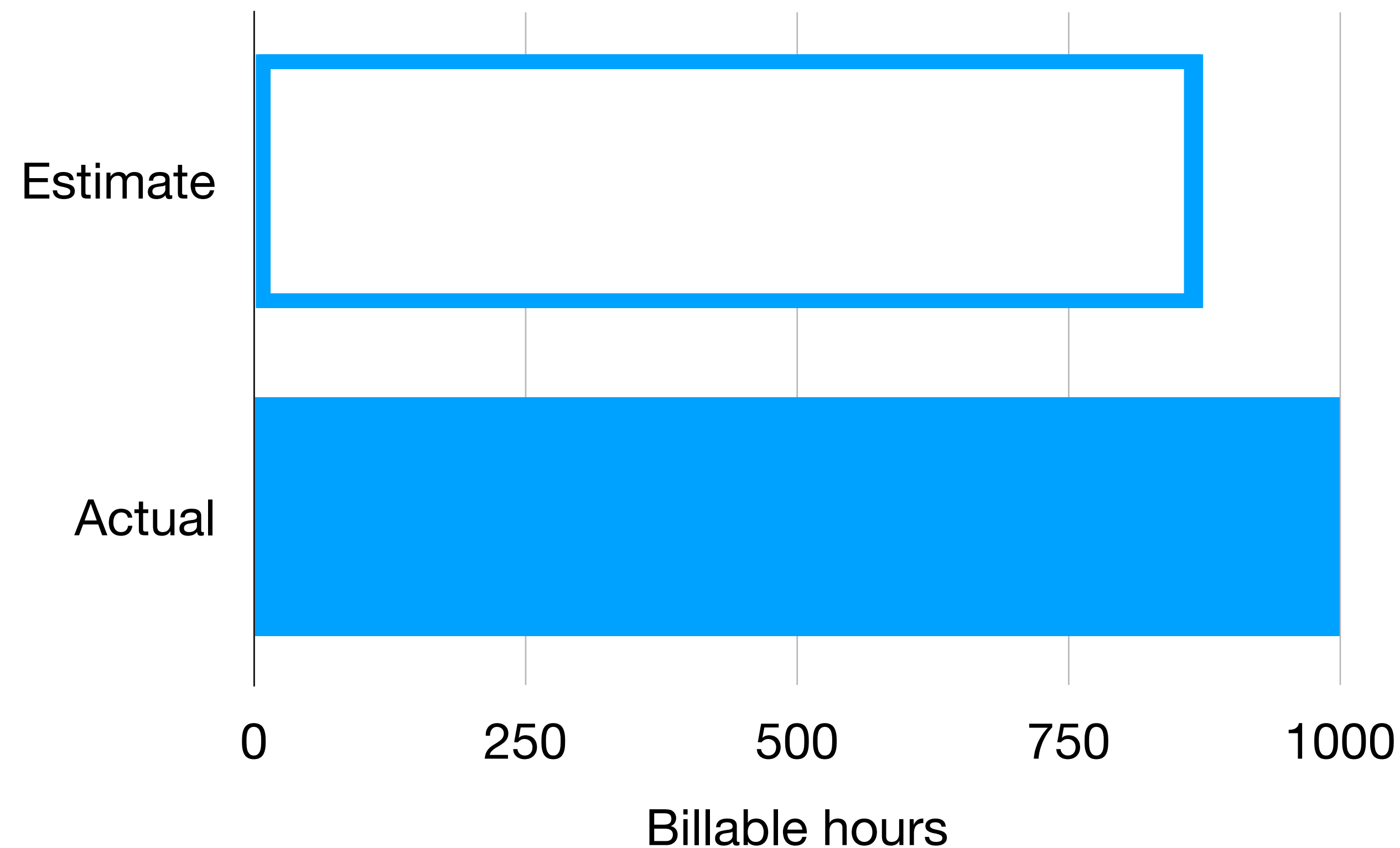
Dynamic website = many h

Static website = few h



Project controlling

Compare actual effort with estimated



- During project: How much budget is left?
- After project: What was the marginal return?

Valid reasons to estimate

- Generally: #NoEstimate - "Estimate is waste"
- Compete for contract with estimates from others.
- Do we even have the resources?
- Business case: Is the resulting product/service even worth it?
- Hon your estimation skills when it does not matter so you have them when it matters.

Limitations of project controlling

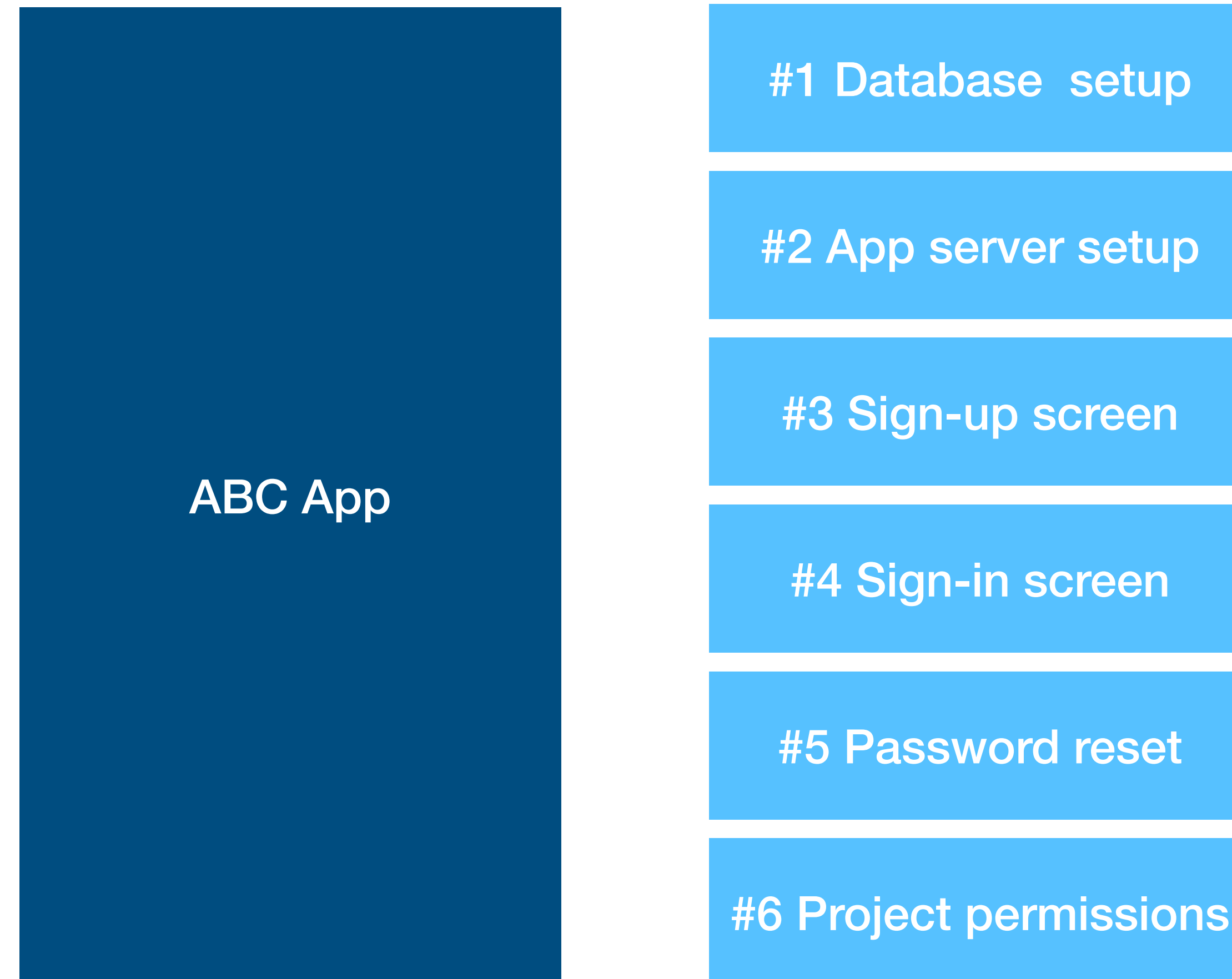
- Few botched tasks can derail project
- Can be hard to pinpoint them
- Not helpful for individual person to improve their estimates

Task tracking and controlling

What is a task?

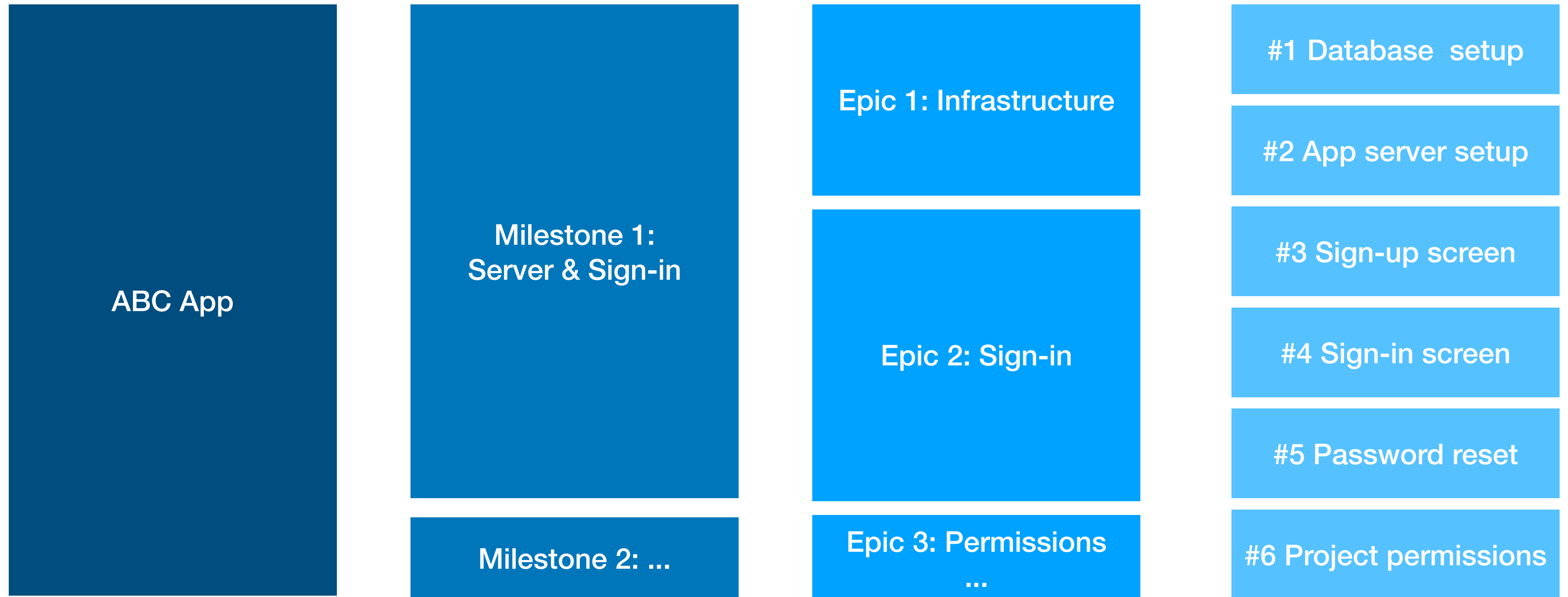
- Something that needs to be done.
- Interchangeable terms:
 - Task
 - Issue
 - Ticket

Split projects in tasks



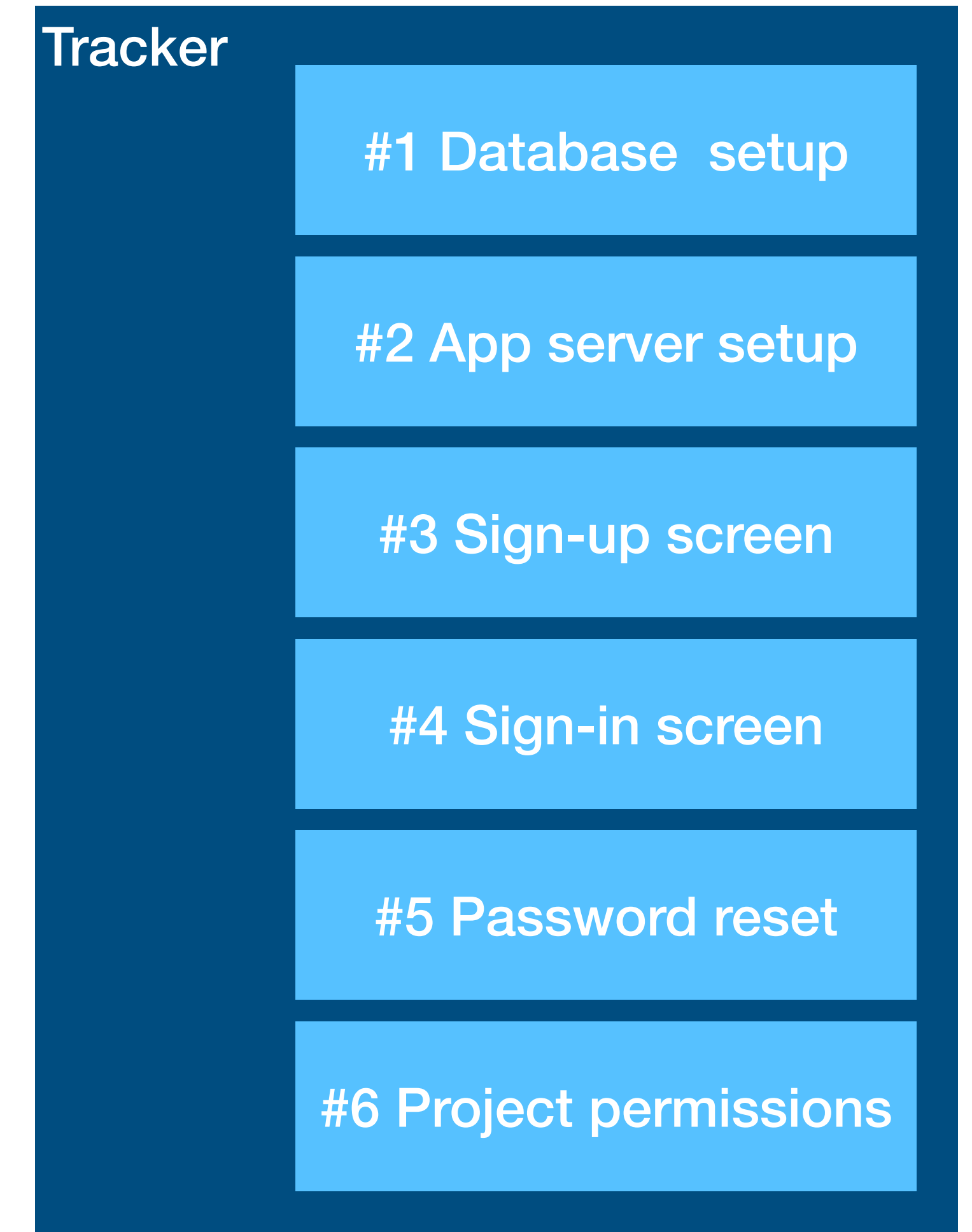
Split projects in tasks

Principle remains even with further hierarchies



Task trackers

- Collect tasks that belong to e.g. same project
- Add metadata like assignee, status
- Have description and links describing what needs to be done
- Ideally:
 - Definition of ready
 - Definition of done
 - Acceptance criteria



Example: Task in task tracker

From <https://github.com/roskakori/pygount/issues/209>

Add generated files by name #190

 Closed
 #203

rosvakori opened on Mar 16, 2025

Last edited by rosvakori · Owner · ...

Story

As user I want files with certain names to be counted as "generated" instead of "unknown" so that I can treat them accordingly.

Examples: poetry.lock, uv.lock, *.g.dart.

Goals

✓

In the command line, the user can specify the names of generated files using the `--generated-names` option, which works similar to `--names-to-skip`.

...

✓

By default, the following files are recognized as "generated":

...

✓

`*.g.dart`. This seems to be a popular convention, but not a official recommendation, though. Regardless, see for example "[Code Generation with Dart & Flutter: The Ultimate Guide](#)".

✓

`*.lock`, for example Python's `uv.lock` or Rust's `Cargo.lock`.

✓

go's `go.sum`.

✓

npm's `package-lock.json` and `npm-shrinkwrap.json`.

✓

pnpm's `pnpm-lock.yaml`.

There is probably many more, but this should be a good starting point.

Estimate: 2.5h

Create sub-issue

Assignees

rosvakori

Labels

enhancement

Projects

Open source projects

Status

✓ Done

Milestone

✓ v3.1.0

Closed on May 27, 2025, 100% complete

Relationships

None yet

Development

#190 Add generated files by name

rosvakori/pygount

v3.1.0

Notifications

Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread

Track time for task

From <https://github.com/roskakori/pygount/issues/209>

Add generated files by name #190

Closed

#203

roskakori opened on Mar 16, 2025

Last edited by roskakori · Owner · ...

Story

As user I want files with certain names to be counted as "generated" instead of "unknown" so that I can treat them accordingly.

Examples: poetry.lock, uv.lock, *.g.dart.

Goals

☒

In the command line, the user can specify the names of generated files using the `--generated-names` option, which works similar to `--names-to-skip`.

...

☒

By default, the following files are recognized as "generated":

...

☒

*.g.dart. This seems to be a popular convention, but not a official recommendation, though. Regardless, see for example "[Code Generation with Dart & Flutter: The Ultimate Guide](#)".

☒

*.lock, for example Python's `uv.lock` or Rust's `Cargo.lock`.

☒

go's `go.sum`.

☒

npm's `package-lock.json` and `npm-shrinkwrap.json`.

☒

pnpm's `pnpm-lock.yaml`.

There is probably many more, but this should be a good starting point.

Estimate: 2.5h

Create sub-issue

Assignees

roskakori

Labels

enhancement

Projects

Open source projects

Status

Done

Milestone

v3.1.0

Closed on May 27, 2025, 100% complete

Relationships

None yet

Development

#190 Add generated files by name

roskakori/pygount

v3.1.0

Notifications

Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

#190 Add generated files by name

pygount-clockodo

Open Source / pygount / Project management

May 26, 2025, 10:43 p.m. 0:06:20

ta

#190 Add generated files by name

#190 Add generated files by name

pygount-clockodo

Open Source / pygount / Development

May 26, 2025, 10 p.m. 0:43:09

ta

#190 Add generated files by name

#190 Add generated files by name

pygount-clockodo

Open Source / pygount / Development

May 26, 2025, 8:36 p.m. 1:23:06

ta

#190 Add generated files by name

24

Blow-up factor

Blow-up factor

Blow-up factor = actual time / estimated time

Blow-up factor

Blow-up factor = actual time / estimated time

Actual: 3.0h
Estimate: 2.5h

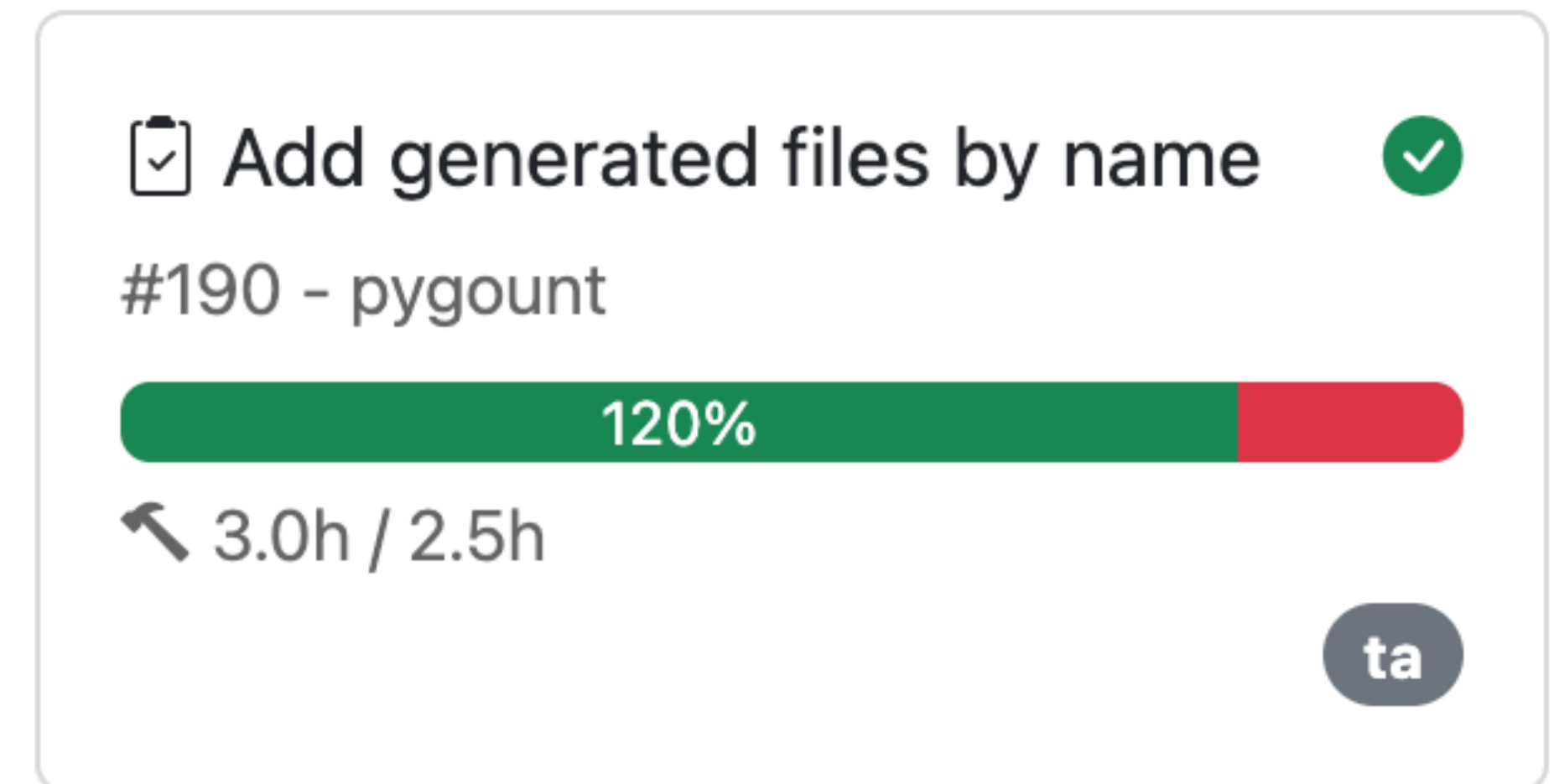
Blow-up factor: $3.0 / 2.5 = 1.2 = 120\%$

Blow-up factor

Blow-up factor = actual time / estimated time

Actual: 3.0h
Estimate: 2.5h

Blow-up factor: $3.0 / 2.5 = 1.2 = 120\%$



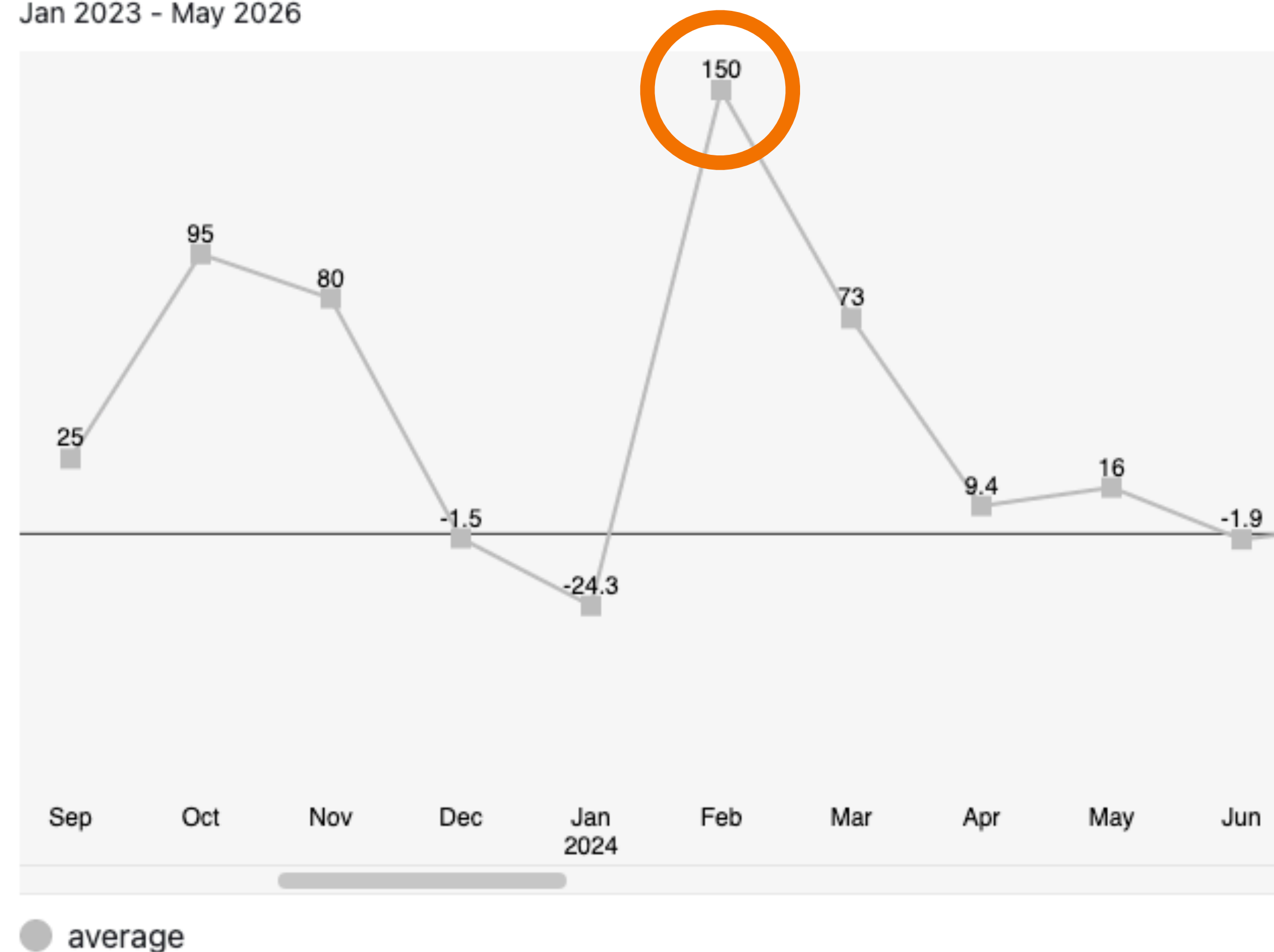
Monitor blow-up factor over time

Average blow-up factor across all tasks per month in percent

Topic: Getting started with docker

Blow-up factor per month in percent

Jan 2023 - May 2026



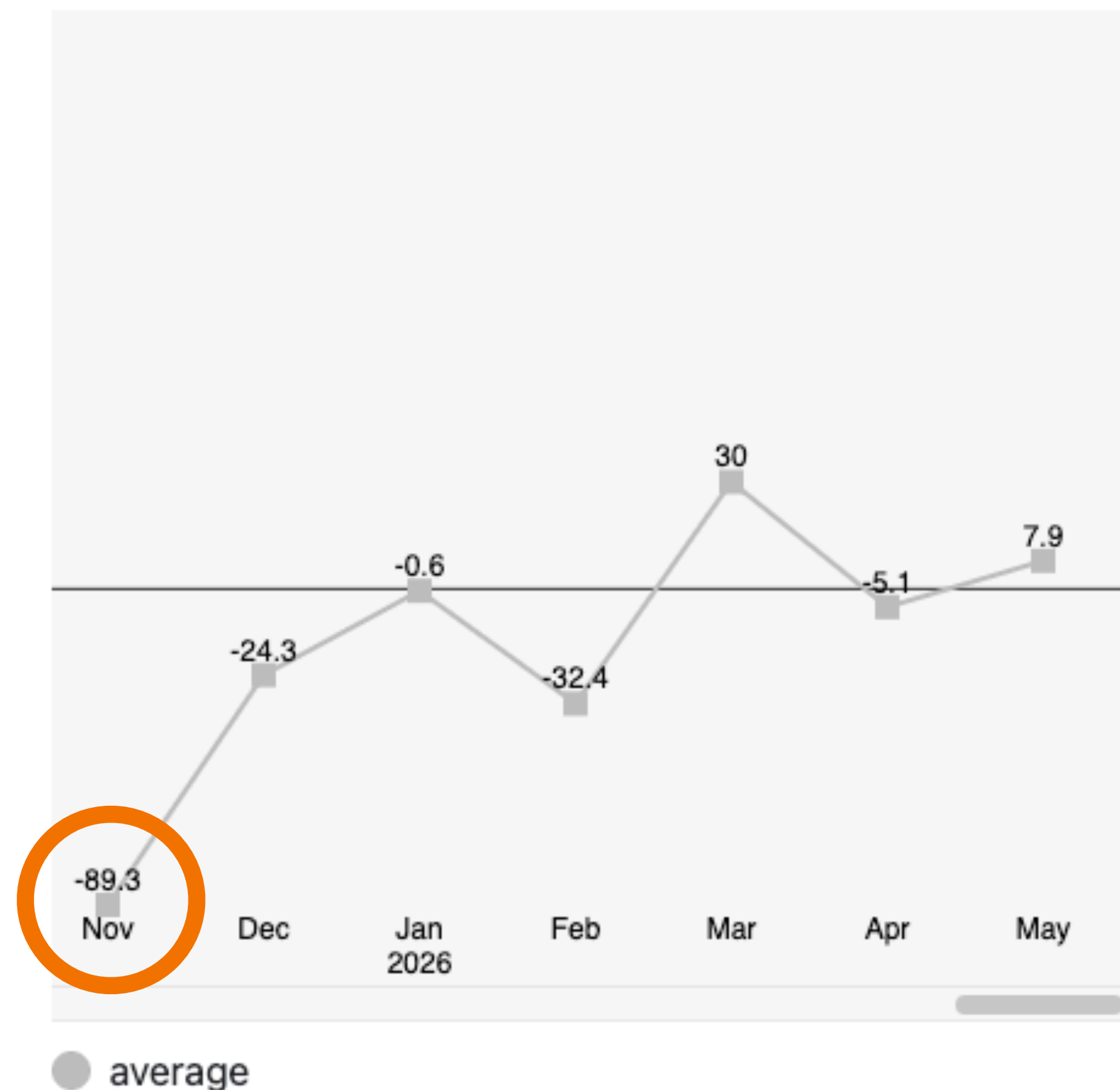
- Dev brain has no experience in estimating ops tasks
- So many ways for docker to not work
- So many things to learn how to trace docker not working

Average blow-up factor across all tasks per month in percent

Topic: Change frontend from Flutter to Django templates

Blow-up factor per month in percent

Jan 2023 - May 2026



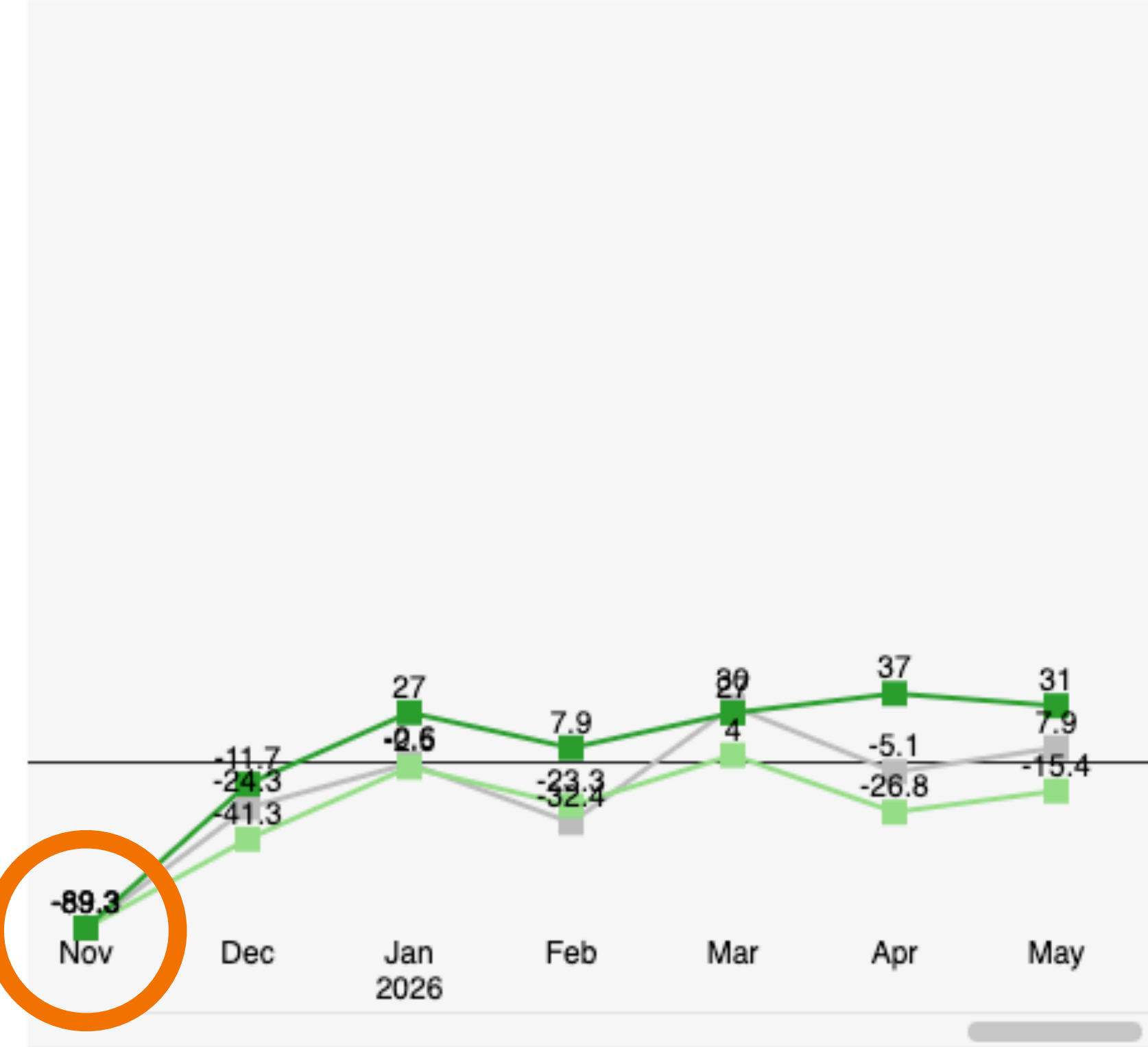
- Architectural change: Move frontend rendering to backend
- Estimation brains needs to adjust that without the need for REST API features can be added faster

Average / Median / 80th percentile

Average is less useful when dealing with outliers

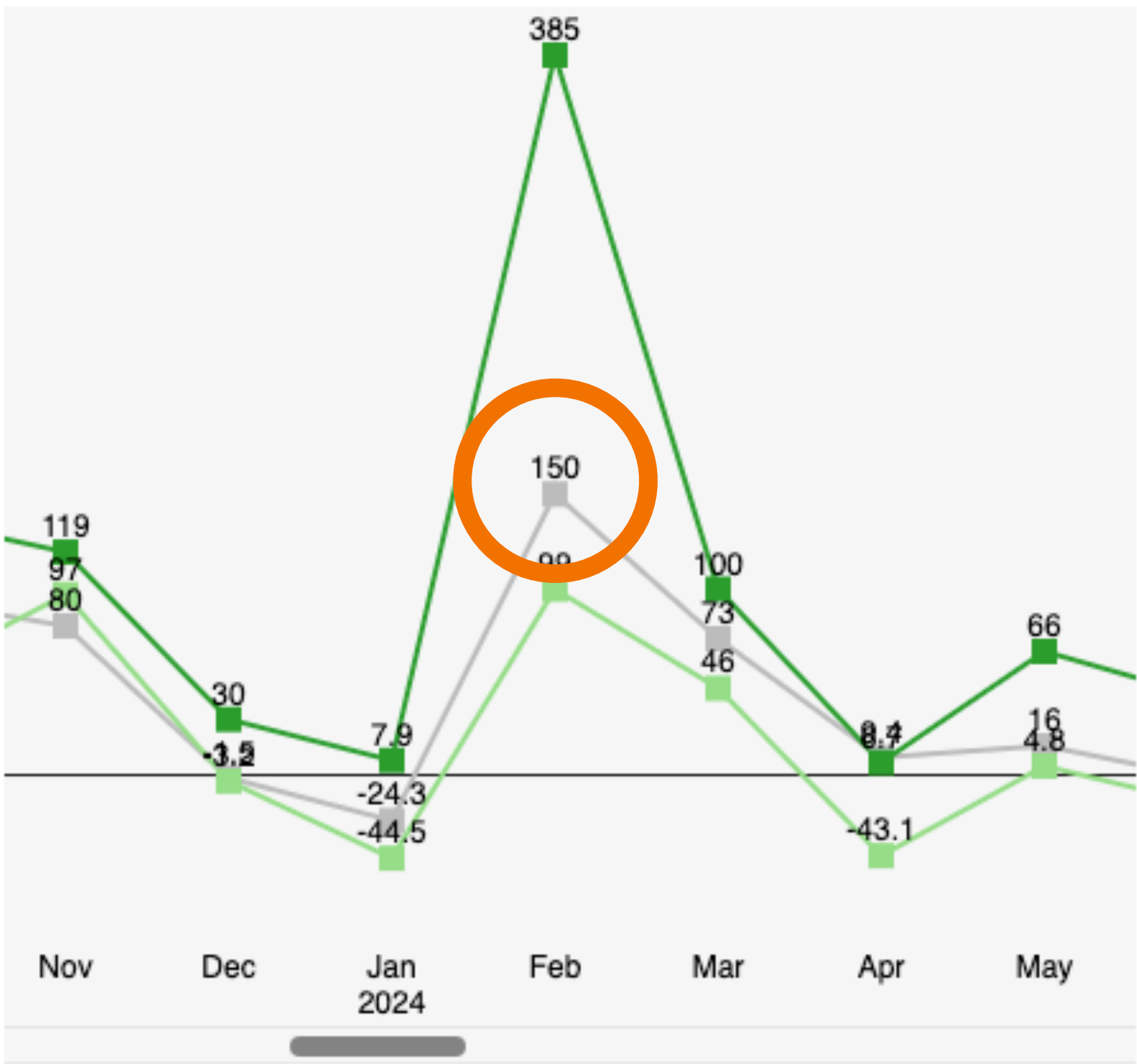
Blow-up factor per month in percent

Jan 2023 - May 2026



Blow-up factor per month in percent

Jan 2023 - May 2026



Reducing the blow-up-factor

Some simple, actionable strategies

- Estimate almost everything, and check blow-up factor
- Split tasks in manageable sizes
- Re-estimate
- For tasks with high blow-up factor, find similarities

Split tasks in manageable sizes

- "Manageable": Aim at 2 - 8h
- Refactor with separate task before actual task
- Fewer risks per task
- Quickly recognize if task is going to blow-up
- Still: Deliver something of value with each task



Image by [Mike Dobe](#) from [Pixabay](#)

Split tasks in manageable sizes

Positive side effects

- Finishing task → feeling of accomplishment
- Less/easier merge conflicts
- Smaller code reviews
- Clearer commit history



Image by [Bernd Joisten](#) from [Pixabay](#)

For tasks with high blow-up factor, find similarities

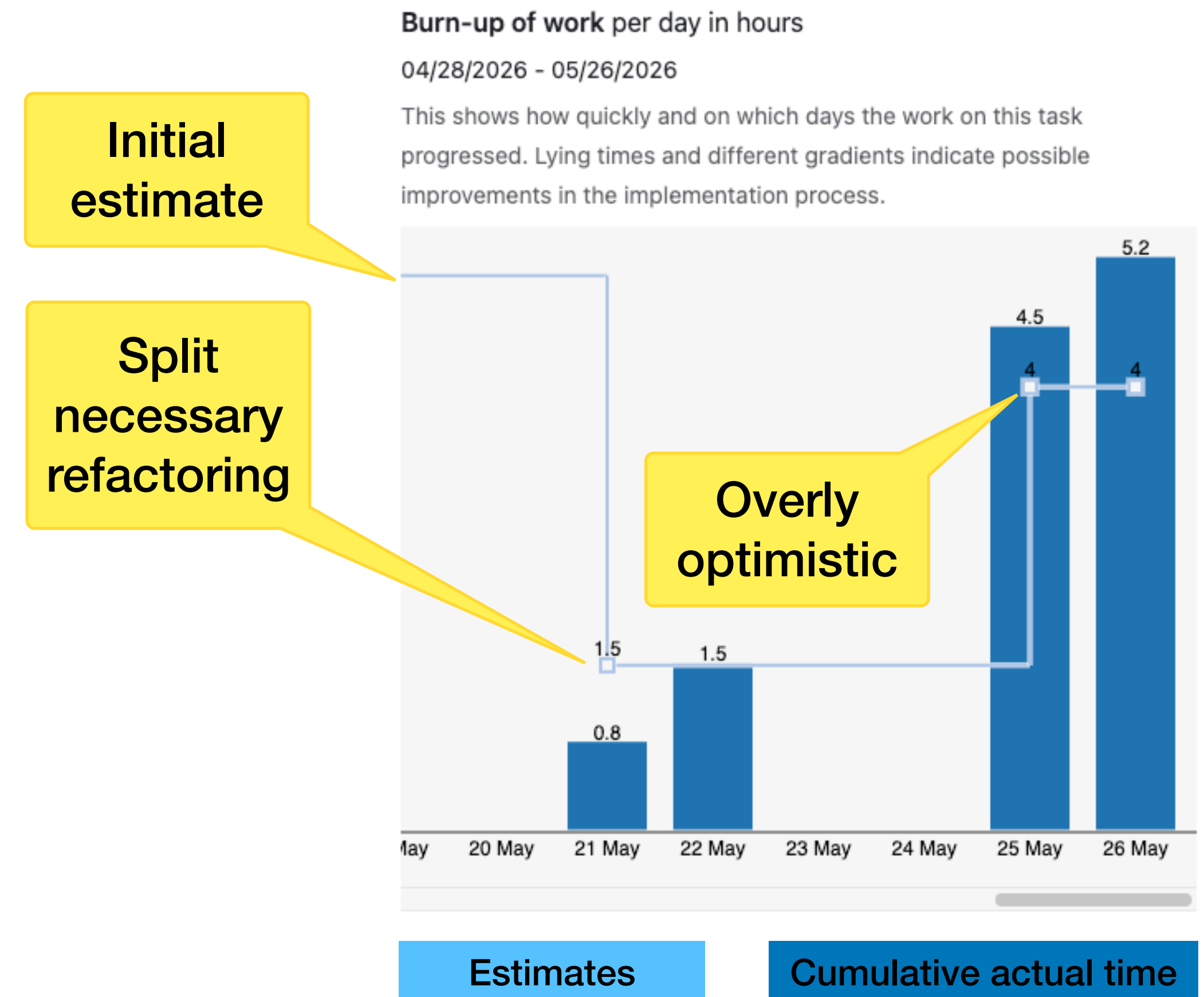
- Scope: frontend, backend, data model, deployment, API
- Technologies: Docker, Python-Rust bridge, JSON Web Token (JWT), ...
- Experience
- Maturity
- Labels & keywords
- People involved



Image by [gmnick](#) from [Pixabay](#)

Re-estimate

- For each estimate of a task, store date
- KPI for development process maturity: Percentage of tasks that needed re-estimate

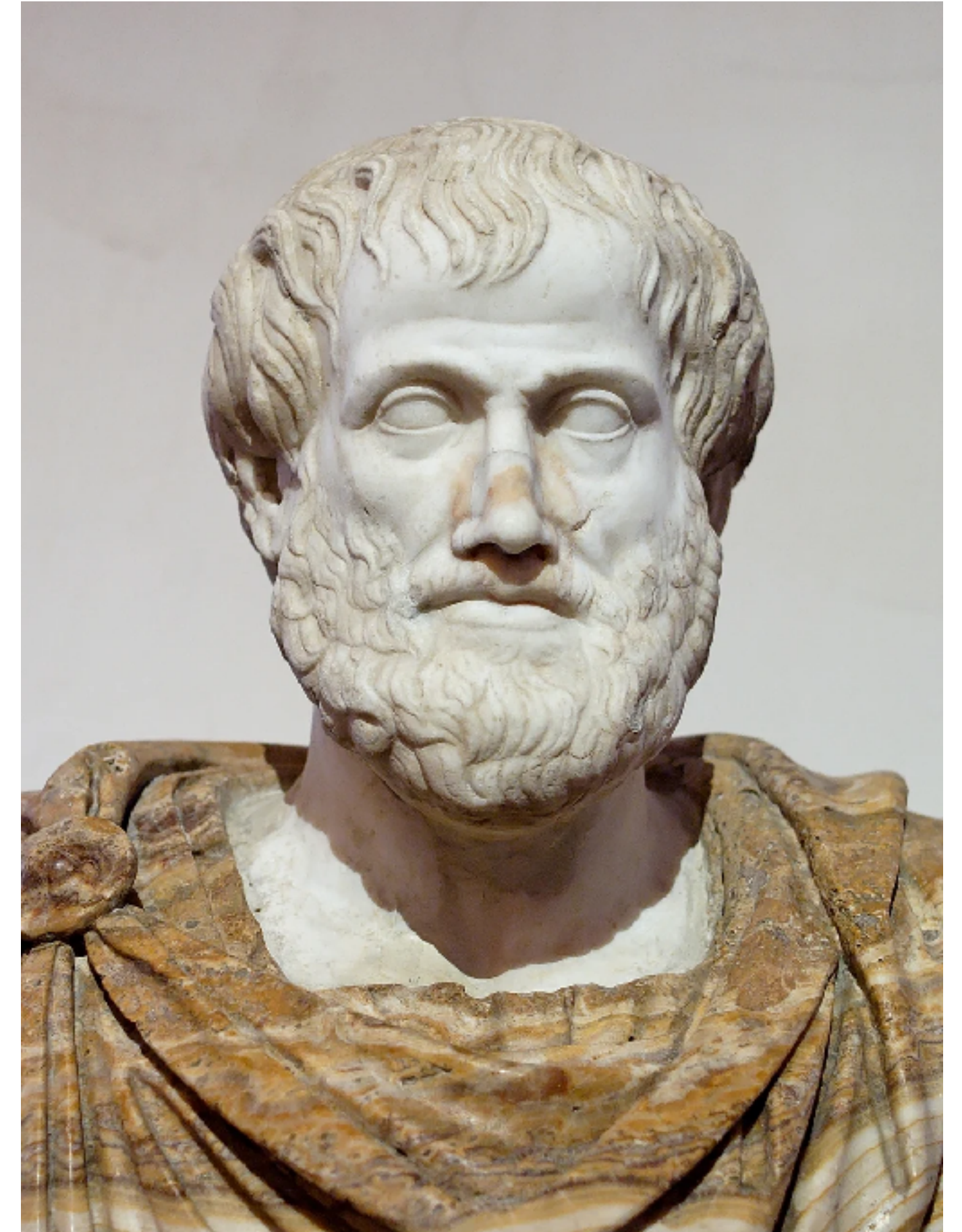


**Current greatest personal
weakness**

Code away!

- Little design and concept for seemingly simple tasks
- Works out most of the time
- Quite regularly only during implementation:
 - Split out into refactoring tasks
 - Understand actual scope and split into multiple tasks
 - Re-estimates

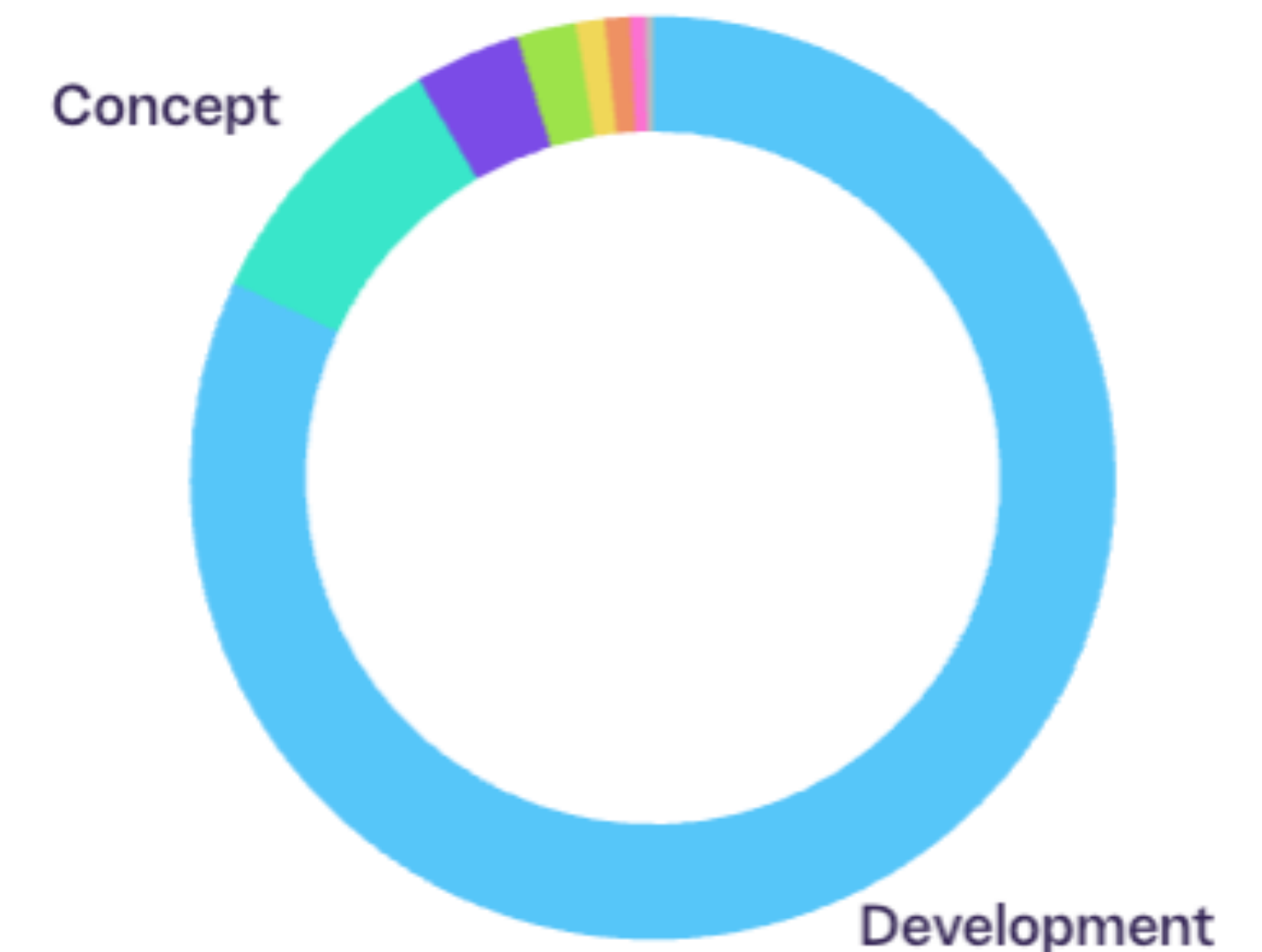
**"Knowing yourself is
the beginning of all
wisdom." (Aristotle)**



By After Lysippos - Jastrow (2006), Public Domain,
<https://commons.wikimedia.org/w/index.php?curid=1359807>

What to do about it?

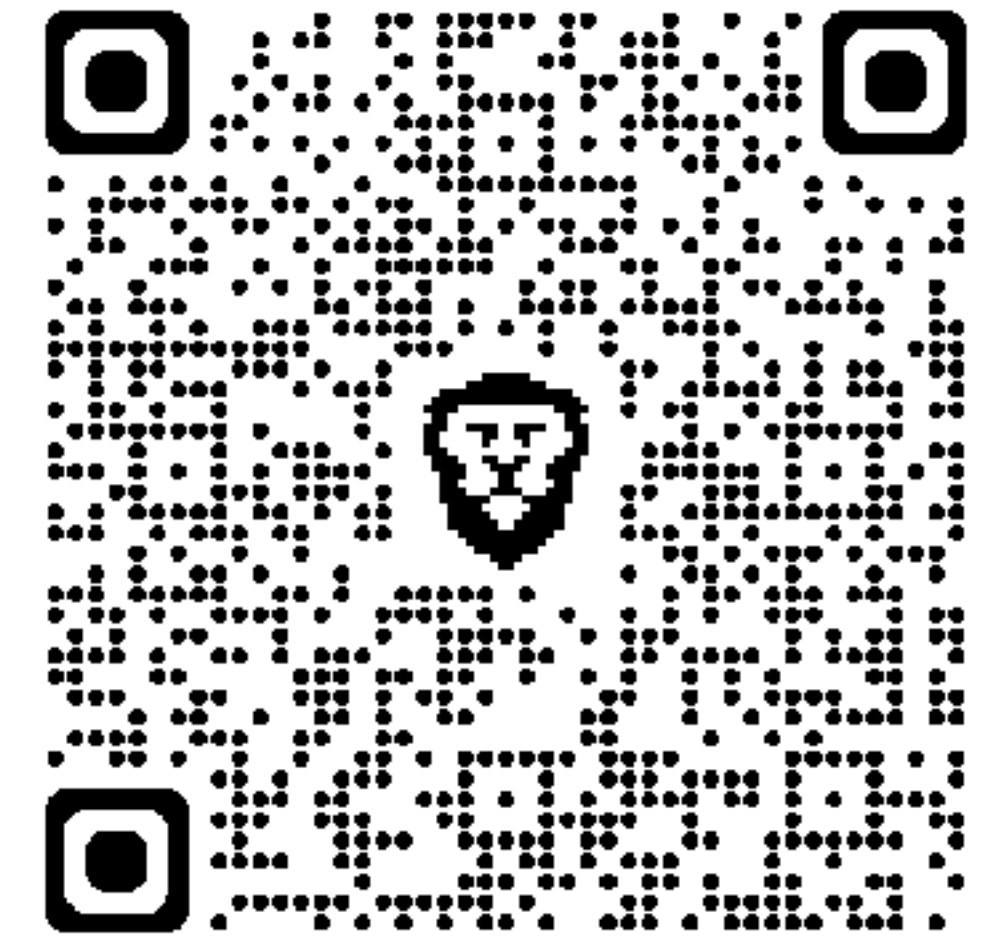
- Add separate time tracker activities for "Concept" and "Development"
- Concept = conceptional work at beginning of a task
- Development = Everything needed to get it done
- Track this to have data
- Find patterns in tasks that went wrong



Summary

Summary

- Estimate and track time per task, not only project.
- Watch the blow-up factor and learn from your mistakes.
- Apply simple strategies to improve your estimates.
- Monitor blow-up factor for actual improvement.



Slides on GitHub